

Theory Supplement

Formal Language Definitions, DFA/NFA Diagrams & Automata-Class Justification

Project Link : <https://tushar-toc-regex.netlify.app/>

Student	Tushar Kaldate	PRN	202401040191
Roll No.	285	Course	2304220T — Theory of Computation
Topic No.	25	Topic Title	Secure File Upload System
Academic Year	2025-26	Batch/Div	D

Name : Tushar Kaldate

PRN : 202401040191

Roll : 285

Div : D

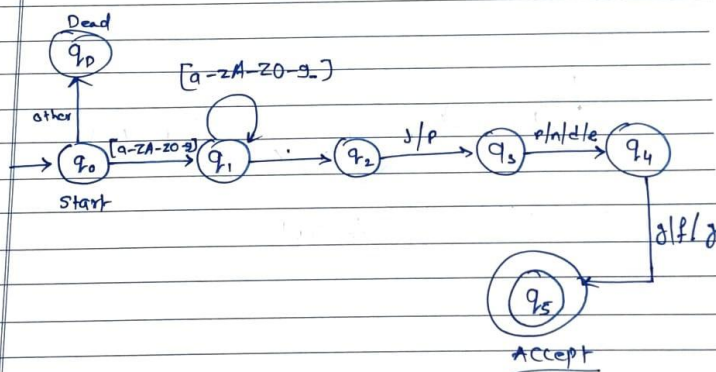
Theory Supplement

1) Regex Design & Pattern Analysis.

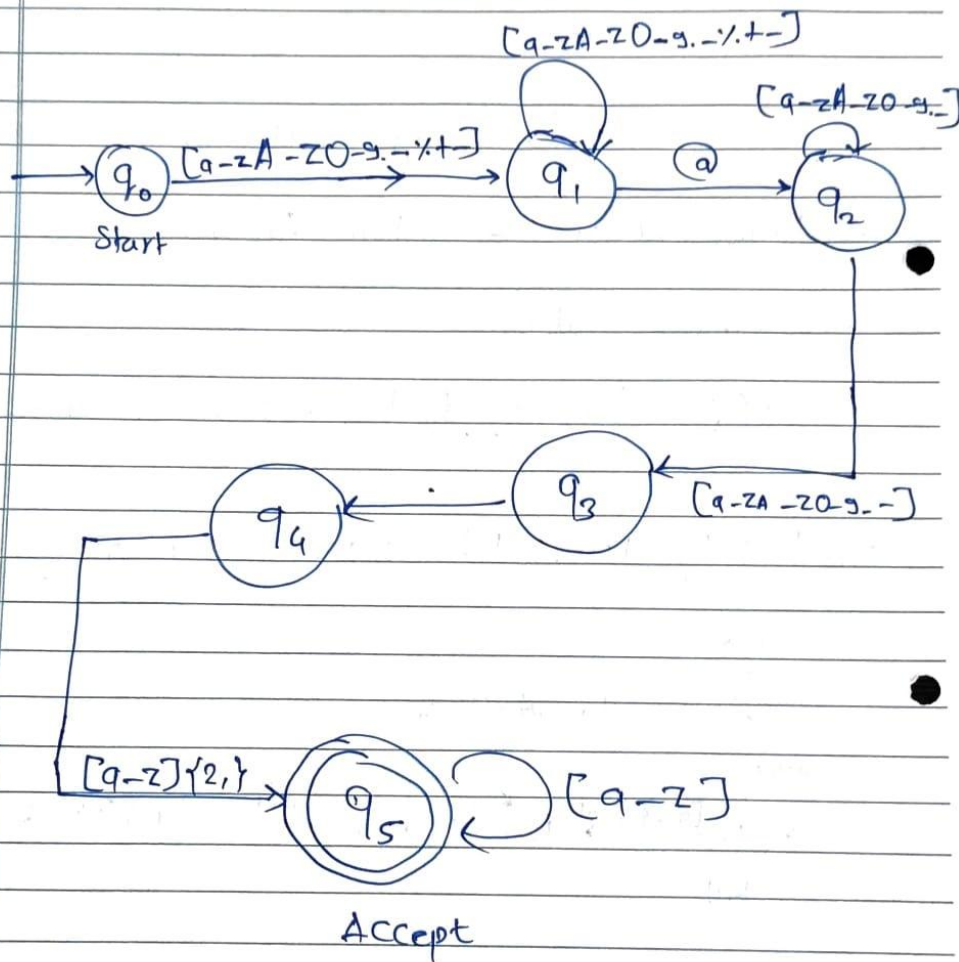
i) Username Validation

$\wedge (? = . \{ 3, 20 \} \$) [a - z A - Z] [a - z A - Z 0 - 9 -] * \$$

* DFA for file name validation



* DFA for Email Validation



This supplement formally describes the regular language recognised by each validation regex, defines the corresponding alphabet, provides DFA state diagrams for two selected fields, and justifies why each language belongs to the class of regular languages as per the Chomsky hierarchy. All patterns exploit closure properties of regular languages.

1

Formal Language Definitions — All 6 Fields

F1: Username

Regex Pattern:	<code>^(?![0-9]+\$)[a-zA-Z0-9_]{3,20}\$</code>
Alphabet (Sigma):	$\Sigma = \{ a-z, A-Z, 0-9, _ \}$
Formal Language:	$L1 = \{ w \text{ in } \Sigma^* \mid 3 \leq w \leq 20, w \text{ in } [a-zA-Z0-9_]^*, w \text{ NOT in } [0-9]^* \}$
Justification:	Finite alphabet and bounded length define a finite-state recognisable set. The all-digits exclusion is the complement of $[0-9]^*$, which is regular (closed under complement). Intersection of two regular languages is regular (Kleene closure property).
Automata Class:	Regular — DFA with O(21) counting states suffices.

F2: Email Address

Regex Pattern:	<code>^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}\$</code>
Alphabet (Sigma):	$\Sigma = \{ a-z, A-Z, 0-9, ., _, \%, +, -, @ \}$
Formal Language:	$L2 = \{ u@d.t \mid u \text{ in } [a-zA-Z0-9._\%+\-]^+, d \text{ in } [a-zA-Z0-9.\-]^+, t \text{ in } [a-zA-Z]{2,} \}$
Justification:	$L2$ is a concatenation of four regular languages: L_{local} , $\{ @ \}$, L_{domain} , $\{ . \}$, L_{tld} . Regular languages are closed under concatenation (Kleene, 1956), so $L2$ is regular.
Automata Class:	Regular — 6-state DFA (see §3).

F3: File Name

Regex Pattern:	<code>^(?!.*\.\w+\.\w+)[a-zA-Z0-9_\-]{1,50}\.(jpg jpeg png pdf docx txt)\$</code>
Alphabet (Sigma):	$\Sigma = \{ a-z, A-Z, 0-9, _, -, . \}$
Formal Language:	$L3 = \{ w.e \mid w \text{ in } [a-zA-Z0-9_\-]{1,50}, e \text{ in } \{ jpg, jpeg, png, pdf, docx, txt \} \}$

Justification:	The extension set {jpg,jpeg,png,pdf,docx,txt} is a finite set (hence regular). The base name is bounded length over a finite alphabet (regular). Concatenation of two regular languages is regular. The double-extension exclusion is a complement+intersection — both closure operations on regular languages.
Automata Class:	Regular — DFA with bounded counting states for base length.

F4: File Extension

Regex Pattern:	<code>^\.(jpg jpeg png pdf docx txt)\$</code>
Alphabet (Sigma):	$\Sigma = \{., a-z, A-Z, 0-9\}$
Formal Language:	$L_4 = \{ .e \mid e \in \{jpg, jpeg, png, pdf, docx, txt\} \}$ subset of Σ^* $L_{bad} = \{ .exe, .bat, .sh, .cmd, .vbs, .ps1, .dll, .php, .js \}$ Accepted: $L_4 = L_{whitelist} \cap \text{complement}(L_{bad})$
Justification:	Both $L_{whitelist}$ and L_{bad} are finite sets — trivially regular. The accepted language is their intersection with complement, both closure operations. A DFA with exactly $ whitelist + blocklist $ paths suffices.
Automata Class:	Regular — finite-set language; single DFA with finitely many accept/reject paths.

F5: File Size (MB)

Regex Pattern:	<code>^\d+(\.\d{1,2})?\$ + JS range: 0 < n <= 100</code>
Alphabet (Sigma):	$\Sigma = \{0-9, .\}$
Formal Language:	$L_5 = \{ w \in \Sigma^* \mid w = d+(\text{dd})?, 0 < \text{parseFloat}(w) \leq 100 \}$ where $d+$ denotes one or more digits
Justification:	The structural regex $\text{d}+(\text{.d}\{1,2\})?$ is a concatenation of regular languages (digit strings and optional decimal suffix). The numeric range constraint is computed in $O(1)$ by the JS engine after the DFA accepts the form — this does not violate regularity since it is a post-recognition semantic check, not a structural language property.
Automata Class:	Regular (structural pattern) — semantic range check is decidable in $O(1)$.

F6: Description

Regex Pattern:	<code>^(?!\\s*\$)[a-zA-Z0-9\\s.,!?'\\-]{10,300}\$</code>
Alphabet (Sigma):	$\Sigma = \{a-z, A-Z, 0-9, \text{space}, ., ,, !, ?, ', -\}$
Formal Language:	$L_6 = \{ w \in \Sigma^* \mid 10 \leq w \leq 300, w \text{ NOT in whitespace-only strings} \}$

Justification: The character class and length bounds define a regular language (DFA with 301 counting states). The whitespace-only exclusion = complement($\backslash s^*$) intersected with L6 — regular languages are closed under complement and intersection.

Automata Class: Regular — DFA with 301 length-counting states suffices.

2 Closure Properties of Regular Languages Used

The following closure properties (all provable from the Myhill-Nerode theorem and DFA construction) are exploited throughout this project:

Property	How Used in This Project	Field(s)
Concatenation <i>L1 . L2 is regular</i>	Email = L_local . {@} . L_domain . {.} . L_tld File Name = L_base . {.} . L_ext	F2, F3
Union <i>L1 UNION L2 is regular</i>	Extension whitelist: (jpg jpeg png pdf docx txt) — finite union	F3, F4
Kleene Star <i>L* is regular</i>	[a-zA-Z0-9_]* used in base-name and local-part sub-patterns	F1, F2, F3
Complement <i>Complement(L) is regular</i>	Blocking all-digits in F1; blocking whitespace-only in F6; blocking bad extensions in F4	F1, F4, F6
Intersection <i>L1 INTERSECT L2 is regular</i>	F4 accepted = L_whitelist INTERSECT Complement(L_blocklist)	F4
Length-bounded <i>{ w a <= w <= b } is regular</i>	Username {3,20}, File Name {1,50}, Description {10,300}	F1, F3, F6

3 DFA State Diagrams

DFA diagrams are provided for two fields as required. Each diagram shows states (circles), transitions (labelled arrows), the start state (gold arrow), and accepting states (double-circle, green). Dead/trap states are implied for all invalid transitions.

DFA 1 — File Name Validation

Pattern: `^[a-zA-Z0-9_-]{1,50}\.(jpg|jpeg|png|pdf|docx|txt)$`

States: q0 (start), q1 (reading base-name chars), q2 (dot seen), q3 (reading extension), q4 (accept); plus implicit dead state for invalid transitions.

Name : Tushar Kaldade

PRN : 202401040191

Roll : 285

Div : D

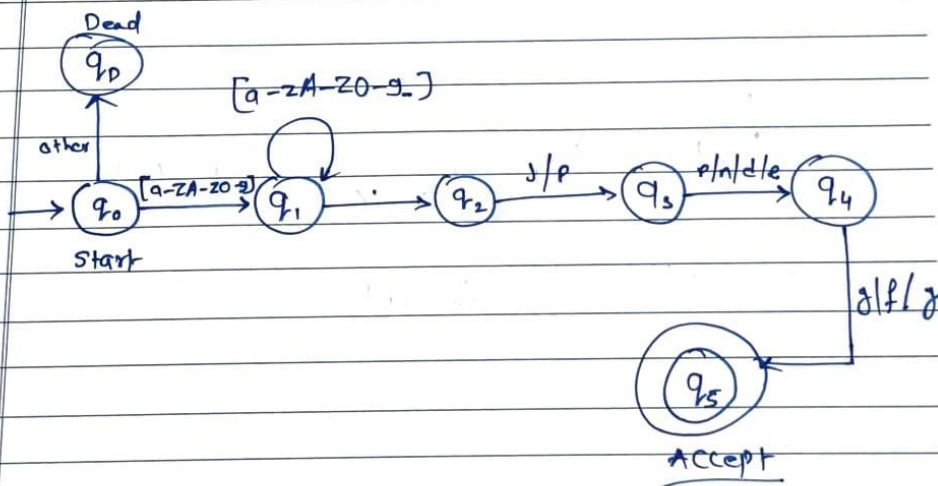
Theory Supplement

1) Regex Design & Pattern Analysis.

i) Username Validation.

$\wedge (? = . \{ 3, 20 \} \$) [a - z A - Z] [a - z A - Z 0 - 9 -] * \$$

* DFA for file name validation

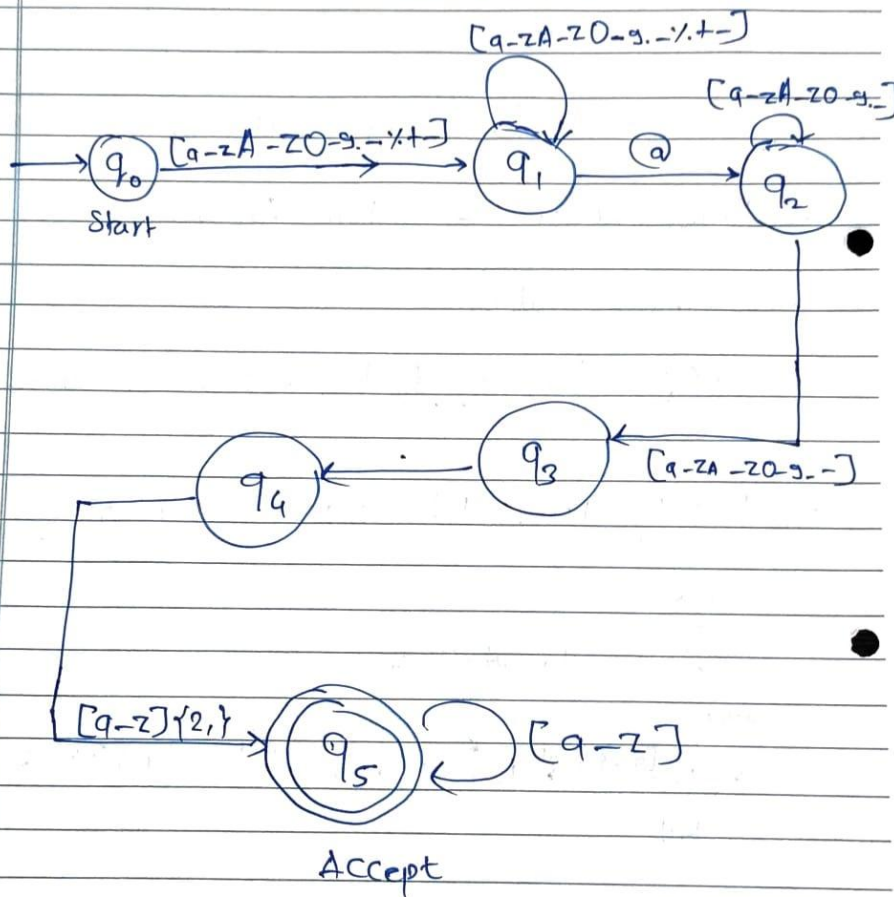


DFA 2 — Email Address Validation

Pattern: $\wedge [a - z A - Z 0 - 9 . _ \% + \backslash -] + @ [a - z A - Z 0 - 9 . \backslash -] + \backslash . [a - z A - Z] \{ 2, \} \$$

States: q0 (start), q1 (local part), q2 (@ seen), q3 (domain part), q4 (dot seen), q5 (TLD reading), q6 (accept).

* DFA For Email Validation



JavaScript regex lookaheads are zero-width assertions — they assert a condition at the current position without consuming input. They do not add expressive power beyond regular languages; instead they are syntactic shortcuts for intersection/complement operations:

Lookahead Used	Field	Formal Equivalent	Closure Property
<code>(?![0-9]+\$)</code>	Username	$L \text{ INTERSECT Complement}([0-9]^*)$	Complement + Intersection
<code>(?!.*\.\w+\.\w+)</code>	File Name	$L \text{ INTERSECT Complement}(\text{double-ext})$	Complement + Intersection
<code>(?!\\s*\$)</code>	Description	$L \text{ INTERSECT Complement}(\backslash s^*)$	Complement + Intersection
<code>(?!\\.(exe bat ..)\$)</code>	Ext Type	$L \text{ INTERSECT Complement}(L_{\text{bad}})$	Complement + Intersection

5 Kleene's Theorem & Regularity Justification

Theorem (Kleene, 1956): A language L is regular if and only if it can be described by a regular expression.

Equivalence Chain:

L is regular

IF AND ONLY IF there exists a DFA M such that $L(M) = L$

IF AND ONLY IF there exists an NFA N such that $L(N) = L$

IF AND ONLY IF there exists a regular expression r such that $L(r) = L$

Application: All six validation patterns in this project are regular expressions over finite alphabets. By Kleene's theorem, each corresponds to a DFA with finitely many states. None of the validated languages require balanced nesting or unbounded counting — properties that would require a PDA (Context-Free) or TM (Recursively Enumerable). Therefore, the Pumping Lemma for regular languages is satisfied for all L_1 – L_6 , confirming membership in REG (the class of regular languages).